

Relatório: Processamento de Lista Encadeada com OpenMP Tasks (Tarefa 7)

1. Introdução

A paralelização de estruturas de dados irregulares, como listas encadeadas, apresenta um desafio particular, pois o acesso aos elementos precisa ser estritamente sequencial (`current = current->next`). O padrão OpenMP resolve esse problema com o modelo de *Tasks* (tarefas). A diretiva `#pragma omp task` permite que um thread encapsule uma unidade de trabalho que será executada por qualquer thread disponível dentro do agrupamento paralelo, de maneira assíncrona.

Neste relatório, implementou-se um programa em C que percorre uma lista encadeada de arquivos fictícios, alocando o processamento de cada nó para tarefas OpenMP.

2. Metodologia e Implementação C

O código C possui duas abordagens distintas para demonstrar os efeitos do aninhamento do loop:

1. **Sem sincronização estrutural:** O laço de travessia e criação de tasks é colocado diretamente dentro da região `#pragma omp parallel`.
2. **Com controle estrutural (`omp single`):** O laço é encapsulado pela diretiva `#pragma omp single`.

A função `omp_get_thread_num()` foi utilizada dentro do bloco de tarefa para expor o ID da thread que de fato executou o processamento.

Código-Fonte Completo

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <omp.h>

// Estrutura do nó da lista encadeada
typedef struct Node {
    char filename[256];
    struct Node* next;
} Node;

// Função para adicionar um nó ao final da lista
void append(Node** head, const char* name) {
    Node* new_node = (Node*)malloc(sizeof(Node));
    strcpy(new_node->filename, name);
    new_node->next = NULL;

    if (*head == NULL) {
        *head = new_node;
        return;
    }
}
```

```

    Node* current = *head;
    while (current->next != NULL) {
        current = current->next;
    }
    current->next = new_node;
}

// Função para liberar a memória da lista
void free_list(Node* head) {
    Node* current = head;
    while (current != NULL) {
        Node* next = current->next;
        free(current);
        current = next;
    }
}

int main() {
    Node* head = NULL;

    // Criação da lista com arquivos fictícios
    append(&head, "documento_A.txt");
    append(&head, "imagem_B.png");
    append(&head, "planilha_C.csv");
    append(&head, "codigo_D.c");
    append(&head, "log_E.txt");

    printf("--- Execucao 1: Regiao paralela SEM sincronizacao de criacao --
\n");
    #pragma omp parallel
    {
        Node* current = head;
        while (current != NULL) {
            // CUIDADO: Todas as threads irão iterar a lista e gerar tasks
            simultaneamente
            #pragma omp task firstprivate(current)
            {
                printf("[Incorreto] Thread %d processando arquivo: %s\n",
omp_get_thread_num(), current->filename);
            }
            current = current->next;
        }
    }

    printf("\n--- Execucao 2: Regiao paralela COM #pragma omp single ---
\n");
    #pragma omp parallel
    {
        // CORRETO: Apenas UMA thread percorre a lista e gera as tarefas
        #pragma omp single
        {
            Node* current = head;
            while (current != NULL) {
                #pragma omp task firstprivate(current)

```

```

        {
            printf("[Correto] Thread %d processando arquivo: %s\n",
omp_get_thread_num(), current->filename);
        }
        current = current->next;
    }
} // A barreira implícita garante que as tarefas finalizem antes de
sair do pragma single
}

free_list(head);
return 0;
}

```

3. Reflexões e Análise dos Resultados

Ao executarmos o código num processador multicore (exemplo de execução local constou com 8 threads lógicas nativas), observou-se as seguintes ocorrências frente às perguntas propostas na descrição da tarefa:

Todos os nós foram processados? Algum foi processado mais de uma vez ou ignorado?

- **Na Abordagem 1 (Sem `omp single`):** Todos os nós foram processados, no entanto, **foram processados dezenas de vezes**. Se existirem T threads e N elementos, serão criadas dinamicamente e enfileiradas $ST \times N$ tarefas. Nenhuma foi ignorada, contudo, a completa redundância torna a abordagem inviável logicamente.
- **Na Abordagem 2 (Com `omp single`):** Cada um dos elementos originais (5 nós) foi processado **exatamente uma única vez**.

O comportamento muda entre execuções?

Sim, muda substancialmente a cada iteração de execução. As *tasks* são enfileiradas pela API no *Thread Pool* global da região paralela. Como o sistema operacional aplica escalonamento preemptivo concorrente entre as threads, as tarefas pendentes na pool de execução (`task queue`) do OpenMP são distribuídas para diferentes threads de forma assíncrona, dependendo de quais ficam "livres" primeiro. Isso altera não só a sequência do "print", como também intercala livremente quais IDs de Threads processam cada elemento, tornando o resultado da ordem imprevisível e não-determinístico (o que é o esperado do modelo paralelo puro).

Como garantir que cada nó seja processado uma única vez e por apenas uma tarefa?

Para assegurar a unicidade do processamento de uma estrutura sequencial que demanda um ponteiro, é vital **centralizar a geração das tarefas**.

- A técnica definitiva é englobar o laço `while(current != NULL)` iterador com o `#pragma omp single`.
- Desta forma, apenas **uma thread arbitrária** assume a responsabilidade de varrer a lista sequencialmente e invocar `#pragma omp task`.
- Conforme ela distribui as tarefas, todas as outras threads do `omp parallel` (e eventualmente a geradora também, nos seus períodos ociosos) começam instantaneamente a retirar as tarefas já

formadas da fila e rodar o trabalho em paralelo.

- A cláusula `firstprivate(current)` no bloco interno também é obrigatória. Como a thread formadora continua alterando a variável "current" rapidamente no loop (avançando os ponteiros), a task gerada precisa copiar para seu escopo interno privado para qual ponteiro ela apontava originalmente no exato microssegundo da sua criação, de forma estanque e salva.